aa274a/hw1/P1_astar.py

```python
from __future__ import annotations

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from utils import plot_line_segments

class AStar(object):
    """Represents a motion planning problem to be solved using A*"""

    def __init__(self, statespace_lo, statespace_hi, x_init, x_goal, occupancy: DetOccupancyGrid2D, resolution=1,
                 heuristic="l2", heuristic_weight=1.0):
        self.statespace_lo = statespace_lo         # state space lower bound (e.g., [-5, -5])
        self.statespace_hi = statespace_hi         # state space upper bound (e.g., [5, 5])
        self.occupancy = occupancy                 # occupancy grid (a DetOccupancyGrid2D object)
        self.resolution = resolution               # resolution of the discretization of state space (cell/m)
        self.heuristic_type = heuristic            # norm used by the heuristic: "l1", "l2", or "linf"
        self.heuristic_weight = heuristic_weight   # weight w >= 1 on the heuristic (w > 1 gives weighted A*)
        self.x_offset = x_init
        self.x_init = self.snap_to_grid(x_init)    # initial state
        self.x_goal = self.snap_to_grid(x_goal)    # goal state

        self.closed_set = set()    # the set containing the states that have been visited
        self.open_set = set()      # the set containing the states that are candidate for future expansion

        self.est_cost_through = {}  # dictionary of the estimated cost from start to goal passing through state (often called f score)
        self.cost_to_arrive = {}    # dictionary of the cost-to-arrive at state from start (often called g score)
        self.came_from = {}         # dictionary keeping track of each state's parent to reconstruct the path

        self.open_set.add(self.x_init)
        self.cost_to_arrive[self.x_init] = 0
        self.est_cost_through[self.x_init] = self.heuristic(self.x_init)

        self.path = None        # the final path as a list of states

    def is_free(self, x):
        """
```

```
        Checks if a give state x is free, meaning it is inside the bounds of the map and
        is not inside any obstacle.
        Inputs:
            x: state tuple
        Output:
            Boolean True/False
        Hint: self.occupancy is a DetOccupancyGrid2D object, take a look at its methods for what might be
              useful here
        """
        ########## Code starts here ##########
        collision_free = self.occupancy.is_free(x)

        in_bounds = (
            x[0] >= self.statespace_lo[0] and
            x[1] >= self.statespace_lo[1] and
            x[0] < self.statespace_hi[0] and
            x[1] < self.statespace_hi[1]
        )

        return collision_free and in_bounds
        ########## Code ends here ##########

    def distance(self, x1, x2):
        """
        Computes the Euclidean distance between two states. This is the cost of
        traveling along the edge between two neighboring states, and it is used
        for the cost-to-arrive. It should NOT depend on self.heuristic_type.
        Inputs:
            x1: First state tuple
            x2: Second state tuple
        Output:
            Float Euclidean distance

        HINT: This should take one line. Tuples can be converted to numpy arrays using np.array().
        """
        ########## Code starts here ##########
        return np.linalg.norm(np.asarray(x1) - np.asarray(x2))
        ########## Code ends here ##########

    def heuristic(self, x):
        """
        Computes the weighted heuristic estimate of the cost-to-go from state x
        to self.x_goal, i.e. w * ||x - x_goal||, where w = self.heuristic_weight
        and the norm is selected by self.heuristic_type:
            "l1":   sum of absolute coordinate differences
            "l2":   Euclidean norm
            "linf": largest absolute coordinate difference
        Input:
```

```
            x: state tuple
        Output:
            Float heuristic value

        HINT: np.linalg.norm accepts an `ord` argument. Raise a ValueError for
              an unrecognized self.heuristic_type.
        """
        ########## Code starts here ##########
        if self.heuristic_type not in ["l1", "l2", "linf"]:
            raise ValueError(f"Unrecognized heuristic_type: {self.heuristic_type}")

        ord = {"l1": 1, "l2": 2, "linf": np.inf}

        return self.heuristic_weight * np.linalg.norm(np.asarray(self.x_goal) - np.asarray(x), ord=ord[self.heuristic_type])
        ########## Code ends here ##########

    def snap_to_grid(self, x):
        """ Returns the closest point on a discrete state grid
        Input:
            x: tuple state
        Output:
            A tuple that represents the closest point to x on the discrete state grid
        """
        return (
            self.resolution * round((x[0] - self.x_offset[0]) / self.resolution) + self.x_offset[0],
            self.resolution * round((x[1] - self.x_offset[1]) / self.resolution) + self.x_offset[1],
        )

    def get_neighbors(self, x):
        """
        Gets the FREE neighbor states of a given state x. Assumes a motion model
        where we can move up, down, left, right, or along the diagonals by an
        amount equal to self.resolution.
        Input:
            x: tuple state
        Ouput:
            List of neighbors that are free, as a list of TUPLES

        HINTS: Use self.is_free to check whether a given state is indeed free.
               Use self.snap_to_grid (see above) to ensure that the neighbors
               you compute are actually on the discrete grid, i.e., if you were
               to compute neighbors by adding/subtracting self.resolution from x,
               numerical errors could creep in over the course of many additions
               and cause grid point equality checks to fail. To remedy this, you
               should make sure that every neighbor is snapped to the grid as it
               is computed.
        """
        neighbors = []
```

```
        ########## Code starts here ##########

        for i in [-1,0,1]:
            for j in [-1,0,1]:
                if i == 0 and j == 0:
                    continue

                candidate = self.snap_to_grid((x[0] + i*self.resolution, x[1] + j*self.resolution))
                if not self.is_free(candidate):
                    continue

                neighbors.append(candidate)
        ########## Code ends here ##########
        return neighbors

    def find_best_est_cost_through(self):
        """
        Gets the state in open_set that has the lowest est_cost_through
        Output: A tuple, the state found in open_set that has the lowest est_cost_through
        """
        return min(self.open_set, key=lambda x: self.est_cost_through[x])

    def reconstruct_path(self):
        """
        Use the came_from map to reconstruct a path from the initial location to
        the goal location
        Output:
            A list of tuples, which is a list of the states that go from start to goal
        """
        path = [self.x_goal]
        current = path[-1]
        while current != self.x_init:
            path.append(self.came_from[current])
            current = path[-1]
        return list(reversed(path))

    def plot_path(self, fig_num=0, show_init_label=True):
        """Plots the path found in self.path and the obstacles"""
        if not self.path:
            return

        self.occupancy.plot(fig_num)

        solution_path = np.asarray(self.path)
        plt.plot(solution_path[:,0],solution_path[:,1], color="green", linewidth=2, label="A* solution path", zorder=10)
        plt.scatter([self.x_init[0], self.x_goal[0]], [self.x_init[1], self.x_goal[1]], color="green", s=30, zorder=10)
```

```
180         if show_init_label:
181             plt.annotate(r"$x_{init}$", np.array(self.x_init) + np.array([.2, .2]),
    fontsize=16)
182         plt.annotate(r"$x_{goal}$", np.array(self.x_goal) + np.array([.2, .2]),
    fontsize=16)
183         plt.legend(loc='upper center', bbox_to_anchor=(0.5, -0.03), fancybox=True,
    ncol=3)
184 
185         plt.axis([0, self.occupancy.width, 0, self.occupancy.height])
186 
187     def plot_tree(self, point_size=15):
188         plot_line_segments([(x, self.came_from[x]) for x in self.open_set if x !=
    self.x_init], linewidth=1, color="blue", alpha=0.2)
189         plot_line_segments([(x, self.came_from[x]) for x in self.closed_set if x !=
    self.x_init], linewidth=1, color="blue", alpha=0.2)
190         px = [x[0] for x in self.open_set | self.closed_set if x != self.x_init and x
    != self.x_goal]
191         py = [x[1] for x in self.open_set | self.closed_set if x != self.x_init and x
    != self.x_goal]
192         plt.scatter(px, py, color="blue", s=point_size, zorder=10, alpha=0.2)
193 
194     def solve(self):
195         """
196         Solves the planning problem using the A* search algorithm. It places
197         the solution as a list of tuples (each representing a state) that go
198         from self.x_init to self.x_goal inside the variable self.path
199         Input:
200             None
201         Output:
202             Boolean, True if a solution from x_init to x_goal was found
203 
204         HINTS:  We're representing the open and closed sets using python's built-in
205                 set() class. This allows easily adding and removing items using
206                 .add(item) and .remove(item) respectively, as well as checking for
207                 set membership efficiently using the syntax "if item in set".
208 
209                 Use self.distance for edge costs (cost-to-arrive) and
210                 self.heuristic for the estimated cost-to-go, as in Algorithm 1
211                 of the handout.
212         """
213         ########## Code starts here ##########
214 
215         # Init sets
216         self.open_set.clear()
217         self.open_set.add(self.snap_to_grid(self.x_init))
218         self.closed_set.clear()
219 
220         # Costs
221         self.cost_to_arrive[self.x_init] = 0
222         self.est_cost_through[self.x_init] = self.heuristic(self.x_init)
223 
```

```
        while len(self.open_set) > 0:
            x_curr = self.snap_to_grid(self.find_best_est_cost_through())
            if x_curr == self.x_goal:
                self.path = self.reconstruct_path()
                return True

            self.open_set.remove(x_curr)
            self.closed_set.add(x_curr)

            for x_neigh in self.get_neighbors(x_curr):
                if x_neigh in self.closed_set:
                    continue

                tentative_cost_to_arrive = self.cost_to_arrive[x_curr] +
self.distance(x_curr, x_neigh)

                if x_neigh not in self.open_set:
                    self.open_set.add(x_neigh)
                elif tentative_cost_to_arrive > self.cost_to_arrive[x_neigh]:
                    continue

                self.came_from[x_neigh] = x_curr
                self.cost_to_arrive[x_neigh] = tentative_cost_to_arrive
                self.est_cost_through[x_neigh] = tentative_cost_to_arrive +
self.heuristic(x_neigh)

        return False
        ########## Code ends here ##########

class DetOccupancyGrid2D(object):
    """
    A 2D state space grid with a set of rectangular obstacles. The grid is
    fully deterministic
    """
    def __init__(self, width, height, obstacles):
        self.width = width
        self.height = height
        self.obstacles = obstacles

    def is_free(self, x):
        """Verifies that point is not inside any obstacles by some margin"""
        for obs in self.obstacles:
            if x[0] >= obs[0][0] - self.width * .01 and \
               x[0] <= obs[1][0] + self.width * .01 and \
               x[1] >= obs[0][1] - self.height * .01 and \
               x[1] <= obs[1][1] + self.height * .01:
                return False
        return True

    def plot(self, fig_num=0):
        """Plots the space and its obstacles"""
```

```
273         fig = plt.figure(fig_num)
274         ax = fig.add_subplot(111, aspect='equal')
275         for obs in self.obstacles:
276             ax.add_patch(
277             patches.Rectangle(
278             obs[0],
279             obs[1][0]-obs[0][0],
280             obs[1][1]-obs[0][1],))
281         ax.set(xlim=(0,self.width), ylim=(0,self.height))
282
```